

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	28.07.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I (M.S.Dinesh / 192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.S.Dinesh

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q33. Debugging Linear Search Code (Incorrect Index Variable Update)

1. Problem Analysis

The given Linear Search code is:

```
def linear_search(arr, key):  
    pos = -1  
    for i in range(len(arr)):  
        if arr[i] == key:  
            pos = i  
    return pos
```

Problem

The function **does not stop** after finding the first occurrence of the target element. Instead, it continues searching through the remaining elements and updates the variable **pos** every time the target is found.

As a result, the function returns the **last occurrence** instead of the **first occurrence**.

Example

Array = [19, 42, 11, 42, 56, 73, 42]

Key = 42

The function updates **pos** as follows:

Step	Index	Element	pos Value
1	0	19	-1
2	1	42	1
3	2	11	1
4	3	42	3
5	4	56	3
6	5	73	3

7	6	42	6
---	---	----	---

Returned value:

6

But the expected answer is:

1

2. Why the Intended Search Behavior is Violated

The statement

```
pos = i
```

keeps updating the index every time the target appears.

Since there is **no break or return statement**, the loop continues until the end of the array.

Therefore, the **last matching index** is returned instead of the **first**.

3. Corrected Solution

The search should stop immediately after finding the first occurrence.

```
def linear_search(arr, key):
    # Traverse each element in the array
    for i in range(len(arr)):

        # Check if current element matches the key
        if arr[i] == key:
            return i    # Return first occurrence immediately

    # Return -1 if key is not found
    return -1
```

4. Optimized Version (Pythonic)

```
def linear_search(arr, key):  
    for index, value in enumerate(arr):  
        if value == key:  
            return index  
    return -1
```

This version is more readable because `enumerate()` provides both the index and value directly.

5. Sample Execution

Input

Array = [19, 42, 11, 42, 56, 73, 42]

Key = 42

Output

First occurrence found at index: 1

6. Time Complexity Analysis

Let **n** be the number of elements.

Case	Explanation	Time Complexity
Best Case	Key found at the first element	O(1)
Average Case	Key found near the middle	O(n)
Worst Case	Key is last or not present	O(n)

Space Complexity

Only a few variables are used.

Auxiliary Space = O(1)

7. Advantages of the Corrected Code

Returns the **first occurrence** correctly.

Stops searching immediately after finding the key.

Avoids unnecessary comparisons.

Improves readability and control flow.

Uses constant extra memory (**O(1)**).

8. Final Correct Code (with Comments)

```
def linear_search(arr, key):
    # Traverse the array from left to right
    for i in range(len(arr)):

        # If the key is found
        if arr[i] == key:
            return i    # Return the first occurrence immediately

    # Key not found
    return -1

# Example
arr = [19, 42, 11, 42, 56, 73, 42]
key = 42

result = linear_search(arr, key)

if result != -1:
    print("First occurrence found at index:", result)
else:
    print("Element not found")
```

Output

```
def linear_search(arr, key):
    # Traverse the array from left to right
    for i in range(len(arr)):

        # If the key is found
        if arr[i] == key:
            return i      # Return the first

    # Key not found
    return -1

# Example
arr = [19, 42, 11, 42, 56, 73, 42]
key = 42

result = linear_search(arr, key)

if result != -1:
    print("First occurrence found at index:",
else:
    print("Element not found")
```

Console

```
Run started
Initializing environment
Installing packages
Running code
First occurrence found at index: 1
Run completed in 2904.5ms
```

Conclusion

The original program returned the **last occurrence** because it kept updating the index variable until the loop finished. The corrected implementation returns immediately when the target is found, ensuring the **first occurrence** is returned. The corrected algorithm has a **best-case time complexity of $O(1)$** , an **average and worst-case time complexity of $O(n)$** , and a **space complexity of $O(1)$** .